Bioinformatics Project

Marco Nicolini, Giorgio Valentini

October 31, 2025

General organization of the exam

The exam is composed by two parts:

1. The student will prepare a software project (see details in section Project) comprising:
   a. The runnable code to replicate the results of the project. This can be a simple python script or a python notebook (in this last case, please provide the .ipynb file). The code should be appropriately commented in order to evaluate if the student understood the code.
   b. A report describing the considered problem, the machine learning approaches applied to solve it, the experimental set-up, the results obtained and a discussion of the results. The report comprises common sections present in a scientific paper[1] (see section Report for further details). The code and report of the project must be delivered by GitHub and notified by e-mail **at least 7 days before the exam**. Questions about the project can be sent via e-mail (*marco.nicolini@unimi.it*).

2. There will be an oral exam. The oral exam will be a discussion about possible errors present in the project (code and report), general questions about the project and it will also evaluate the student knowledge about all the topics explained during the lessons.

Project organization

The project can be done alone or in groups (up to three students per group). The project will be lighter if done alone, while for groups additional mandatory parts are requested. Mandatory parts for groups are marked with the tag **GROUP**. If you are alone but you want to do all the parts, feel free to proceed. When working in groups, each student should contribute equally to the writing of both code and report and any member of the group should have understood the project in its entirety (i.e. not just the parts wrote by the single student)[2]. There is a task tagged as **OPTIONAL**, which is optional for both groups and single students (but in for a penny, in for a pound!). GitHub Repository of the course: `https://github.com/AnacletoLAB/Bioinformatics_practice`.

[1] Have you ever used LaTeX? It is recommended to submit the report using LaTeX (Through Overleaf `https://www.overleaf.com`).

[2] GPT models are indeed powerful instruments, but each line of code and each phrase in your report MUST be well understood.

1 Project description

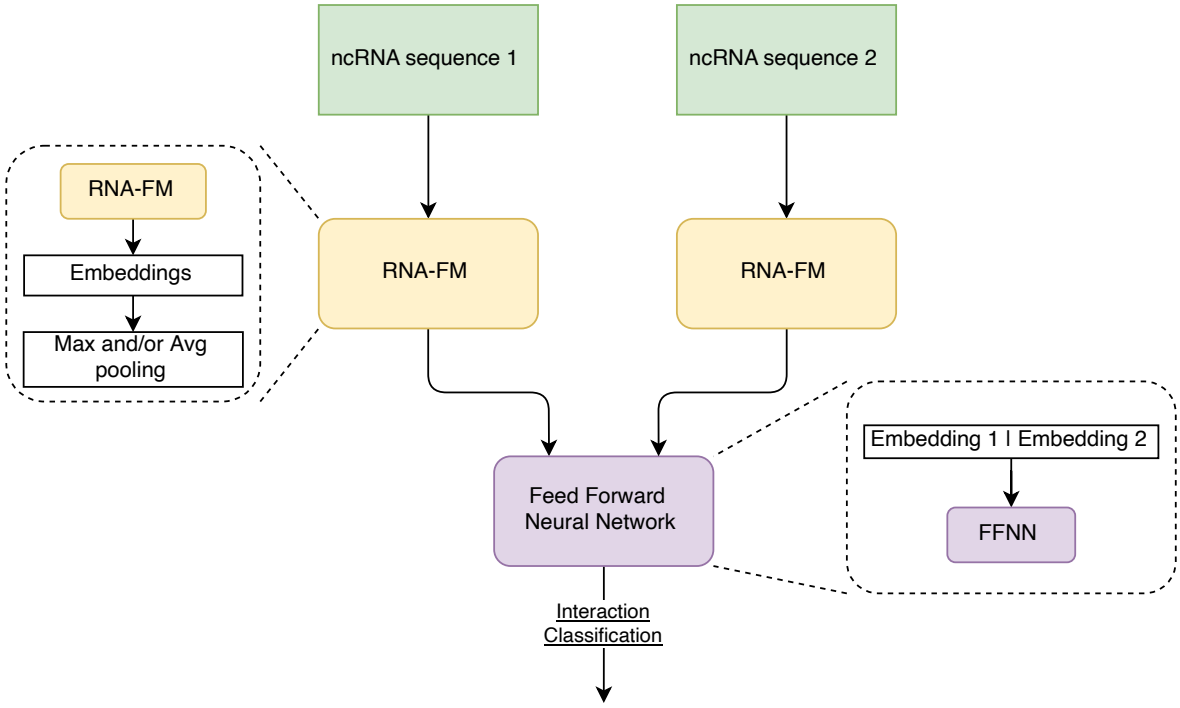


Figure 1: Model architecture schema. In the first stage, a pre-trained ncRNA language model (RNA-FM) transforms ncRNA sequences into dense, embedded representations. These RNA-FM embeddings are subsequently aggregated and processed by a feed-forward neural network to compute interaction probabilities.

Understanding interactions among non-coding RNAs (ncRNAs) is fundamental to analyze post-transcriptional regulation and the architecture of cellular regulatory networks [3]. Experimental mapping of ncRNA–ncRNA interactions, however, remains limited by technical cost and throughput constraints [10]. Computational prediction of these interactions represents a bioinformatics challenge with implications for RNA-based therapeutics.

Traditional computational approaches, such as Minimum Free Energy (MFE) and accessibility-based models, estimate interaction energies between ncRNAs and their targets (e.g., IntaRNA [4]). While effective in specific contexts [8], these models rely on simplifying thermodynamic assumptions and predefined parameters that constrain their generalization capacity. Recent deep learning frameworks have shifted toward data-driven modeling of biomolecular sequences [7, 9, 6], enabling direct inference from raw nucleotide data without handcrafted features.

In this project, we develop a deep learning model for predicting interactions between different ncRNAs (illustrated in Figure 1). The method combines a large pre-trained RNA foundation model (RNA-FM [2]) with a feed-forward neural network (FFNN). RNA-FM, trained on over 23 million ncRNA sequences, generates contextual embeddings that capture sequence and structural regularities. These embeddings are pooled using average and maximum strategies, concatenated, and passed to an FFNN trained to classify interacting versus non-interacting RNA pairs.

This approach departs from energy-based frameworks by leveraging self-supervised embeddings to learn implicit sequence–structure relationships, including non-canonical motifs and contextual dependencies, without relying on explicit secondary structure inputs.

Dataset The interaction data is derived from RNA-KG [1], retaining pairs for which both RNA sequences were available. To ensure compatibility with RNA-FM [2], sequences exceeding 1022 nucleotides must be excluded. The set of ncRNA molecules is denoted as $\mathcal{R} = \{r_i\}$ with associated types $\phi(r_i)$. Interaction identity is defined by unordered pairs $(r_i, r_j) = (r_j, r_i)$.

Data Augmentation and Negative Sampling Each original pair (r_i, r_j) will be expanded into four augmented examples:

$$(r_i, r_j),\ (r_j, r_i),\ (r_i^F, r_j^F),\ (r_j^F, r_i^F),$$

where superscript F indicates sequence reversal. This fourfold augmentation enforces invariance to molecule order and sequence orientation.

Negative pairs are generated by replacing one molecule in each positive pair with another of the same type not known to interact:

$$r' \neq r_i, \quad \phi(r') = \phi(r_j), \quad (r_i, r') \notin \mathcal{P}.$$

For every positive instance, 20 negatives are sampled ($n = 20$) to approximate the imbalance typical of ncRNA interaction data.

RNA-FM Encoder. RNA-FM [2] is a transformer-based encoder comprising 12 bidirectional attention layers. It was trained on over 23 million ncRNA sequences via self-supervised masked-token prediction, yielding embeddings of dimension $H = 640$. Each nucleotide is tokenized individually, producing a hidden representation matrix $\mathbf{X} \in \mathbb{R}^{L \times H}$ for sequence length L.

Tokenization, Feature Extraction, Classifier For each sequence, hidden states are aggregated via average and maximum pooling:

$$\mathbf{e}_{\text{avg}} = \frac{1}{L} \sum_{i=1}^{L} \mathbf{x}_i, \quad \mathbf{e}_{\text{max}} = \max_i \mathbf{x}_i.$$

The final embedding is the concatenation $\mathbf{e} = [\mathbf{e}_{\text{avg}}; \mathbf{e}_{\text{max}}] \in \mathbb{R}^{2H}$. For a pair (r_i, r_j), embeddings are concatenated to form the model input.

The classifier outputs interaction probabilities for each ncRNA pair. The model is trained using binary cross-entropy loss.

Mini-batch Balancing. To counteract the skewed class distribution, each mini-batch includes a fixed ratio of positive and negative samples. Given batch size m, with m_p positives and m_n negatives ($m_p + m_n = m$), positives are sampled with replacement, while negatives are drawn without replacement to ensure diversity.

Experimental Setup The dataset is split into 90% training and 10% test sets; 20% of the training set was used for validation. Early stopping is based on the validation loss and Matthews correlation coefficient [5].

Training is performed using the concatenated average and max pooled embeddings as inputs. Data augmentation is applied to both training and test sets.

1.1 Project steps:

1. Replicate results of miRNA-miRNA interaction prediction with RNA-FM embeddings. Results will not be identical (But similar) because you have to randomly generate the negatives.

2. Describe your assigned interaction Data: How much data do you have before and after filtered for the maximum sequence length? How is the data connected. What is the distribution of the data in terms of sequence length?

3. **GROUP:** Try to visualize the Dataset as a graph, with RNA sequences as nodes and edges the interactions between them, check node degree, and try to generate an image of this graph.

4. Using the same hyperparameters of step 1, train a classifier for your assigned interaction.

5. Hyperparameter-tuning: can you improve AUROC and AUPRC? Keep the same training/test set division. It is not mandatory or required to improve results. Try at least 4 Hyperparameter configurations.

6. **GROUP:** Architectural changes: can you improve AUROC and AUPRC? Keep the same training/test set division. It is not mandatory or required to improve results. Try at least 3 classifier architectures[3]. And at least 4 Hyperparameter configurations for each architecture proposed.

7. **OPTIONAL:** Embeddings (Extracted from the second to last layer of the classifier) visualization with dimensionality reduction methods (U-map, TSNE).

8. Make tables and plots to show the results and discuss them.[4]

[3]Some Examples: a) add or reduce the number of layers of neurons in the fully connected neural network (This counts as one architecture). b) Use Convolutional layer/s. c) Use residual connections. d) Use self/cross-attention layer/s.

[4]AUPRC and AUROC are key metrics, then you can show trainin/validation loss curves, you can set a treshold for the classification, based on results of the validation set (NOT Based on test set results: DATA LEAKAGE), then with that treshold you can compute metrics like accuracy, balanced accuracy, F1, etc.

1.2 Programming language

Since the course was delivered using only Python, Python is the way to go to write the code of the project[5]. The project submission MUST be done through GitHub and notified by email (*marco.nicolini@unimi.it*), so apart from the PDF of the report you need to include in the project repository all the code used for the project[6]. Keep your GitHub repository private, and share it with me (My GitHub username: NicoRota-0).

Possible interactions that will be assigned: circRNA-miRNA, lncRNA-lncRNA, pseudo-miRNA, snoRNA-lncRNA, snoRNA-snoRNA, lncRNA-pseudo.

1.3 Report

The report must contain the following sections:

1. INTRODUCTION: Explain the problem you want to solve. Explore the literature about this topic: Research if there are already available methods in literature that perform the same task (On a different dataset? using a different model?).

2. METHODS: Explain how to reproduce the experiments you performed. Describe Model used, its architecture, Proposed architectural variants, hyperparameters used. Describe the used dataset, and the metrics employed. Explain also the data-preprocessing applied.

3. RESULTS: Present the results of the various approaches using tables and plots. Discuss the obtained results. Note that the results do not necessarily need to be good.

References

[1] Cavalleri, E., Cabri, A., Soto-Gomez, M., Bonfitto, S., Perlasca, P., Gliozzo, J., Callahan, T.J., Reese, J., Robinson, P.N., Casiraghi, E., et al.: An ontology-based knowledge graph for representing interactions involving rna molecules. Scientific Data **11**(1), 906 (2024)

[2] Chen, J., Hu, Z., Sun, S., Tan, Q., Wang, Y., Yu, Q., Zong, L., Hong, L., Xiao, J., Shen, T., et al.: Interpretable rna foundation model from unannotated data for highly accurate rna structure and function predictions. arXiv preprint arXiv:2204.00300 (2022)

[3] Hombach, S., Kretz, M.: Non-coding RNAs: Classification, Biology and Functioning, p. 3–17. Springer International Publishing (2016)

[4] Mann, M., Wright, P.R., Backofen, R.: Intarna 2.0: enhanced and customizable prediction of rna–rna interactions. Nucleic acids research **45**(W1), W435–W439 (2017)

[5] Google Colab (`https://colab.google`) is the easiest way to perform the experiments of this project. In Colab, you can run experiments on dedicated hardware (GPUs/TPUs) without the need to melt your CPU. But winter is coming, so you can always train the models locally to heat a room.

[6] Beware of uploading large files (You don't need to upload RNA-FM model weights)

[5] Matthews, B.W.: Comparison of the predicted and observed secondary structure of t4 phage lysozyme. Biochimica et Biophysica Acta (BBA)-Protein Structure **405**(2), 442–451 (1975)

[6] Nicolini, M., Saitto, E., Jimenez-Franco, R., Cavalleri, E., Galeano, A., Malchiodi, D., Paccanaro, A., Robinson, P., Casiraghi, E., Valentini, G.: Fine-tuning of conditional Transformers improves in silico enzyme prediction and generation. Computational and Structural Biotechnology Journal **27**, 1318–1334 (2025). https://doi.org/10.1016/j.csbj.2025.03.037

[7] Sapoval, N., Aghazadeh, A., Nute, M., et al.: Current progress and open challenges for applying deep learning across the biosciences. Nat Commun **13**(1728) (2022). https://doi.org/http://doi.org/10.1038/s41467-022-29268-7

[8] Umu, S.U., Gardner, P.P.: A comprehensive benchmark of rna–rna interaction prediction tools for all domains of life. Bioinformatics **33**(7), 988–996 (2017)

[9] Valentini, G., Malchiodi, D., Gliozzo, J., Mesiti, M., Soto-Gomez, M., Cabri, A., Reese, J., Casiraghi, E., Robinson, P.: The promises of large language models for protein design and modeling. Frontiers in Bioinformatics **3**(1304099) (2023)

[10] Winkle, M., El-Daly, S.M., Fabbri, M., Calin, G.A.: Noncoding rna therapeutics — challenges and potential solutions. Nature Reviews Drug Discovery **20**(8), 629–651 (Jun 2021). https://doi.org/http://dx.doi.org/10.1038/s41573-021-00219-z